

React JS

সিনিয়র লেভেল রিভিশন — ১৫ মিনিট। Hooks-যুগের React ধরে — class component নিয়ে সময় নষ্ট নয়।

১. React-এর Rendering ও Reconciliation কীভাবে কাজ করে?

Description: setState করলে আসলে কী ঘটে — Virtual DOM diff থেকে DOM update পর্যন্ত। Performance আলোচনার ভিত্তি, তাই সিনিয়রদের এটা পরিষ্কার জানতে হয়।

৬টা Key Point:

1. State change → component function আবার চলে → নতুন React element tree (Virtual DOM) তৈরি হয়
2. Reconciliation: পুরনো ও নতুন tree diff — একই type হলে update, ভিন্ন type হলে subtree unmount + rebuild
3. List-এ **key** এই diff-এর পরিচয়পত্র — index-কে key দিলে reorder/insert-এ state গুলিয়ে যায়
4. Parent re-render হলে সব child default re-render হয় — DOM না বদলালেও function চলে; এটাই optimization-এর মূল টার্গেট
5. React 18: automatic batching — একাধিক setState (event handler-এর বাইরে সহ) এক render-এ; concurrent rendering-এ render interruptible
6. "Re-render হওয়া" ≠ "DOM বদলানো" — DOM write টুকুই দামি; তাই আগে মাপা, তারপর optimize

২. useEffect সঠিকভাবে ব্যবহার ও কমন ভুল

Description: সবচেয়ে ভুল ব্যবহৃত hook। Dependency array, cleanup, আর "কখন useEffect আসলে দরকারই না" — এই তিনটা বুঝলে সিনিয়র উত্তর হয়।

৬টা Key Point:

1. Effect = render-এর পরে বাইরের জগতের সাথে sync (API, subscription, DOM) — data transform effect-এ নয়, render-এই করা
2. Dependency array সং রাখা: ভেতরে ব্যবহৃত সব reactive value থাকবে — lint rule বন্ধ করা মানে বাগ পোষা
3. Cleanup function: subscription/timer/AbortController বন্ধ — নাহলে memory leak আর race condition
4. Fetch-এ race: আগের slow response পরের fast response-কে overwrite করে — cleanup-এ **ignore** flag বা abort
5. কমন ভুল: state থেকে derived value effect + setState দিয়ে বানানো — সরাসরি হিসাব করলেই হয়; "You Might Not Need an Effect"
6. Strict Mode dev-এ effect দুইবার চালায় — cleanup সঠিক থাকলে সমস্যা হয় না; এটা বাগ ধরারই কৌশল

৩. State Management: Local State থেকে Redux পর্যন্ত

Description: কোন state কোথায় রাখবেন — সব Redux-এ ঢালা জুনিয়র লক্ষণ। State-এর ধরন অনুযায়ী সঠিক টুল বাছাই করাই প্রশ্নের আসল উত্তর।

৬টা Key Point:

1. প্রথম নিয়ম: state যত কাছে সম্ভব — এক component-এর হলে **useState**, ভাগ করতে হলে আগে lift up
2. Server data (API response) আসলে client state না — সেটা cache; React Query/SWR এই কাজে Redux-এর চেয়ে ভালো (refetch, stale, invalidation ফ্রি)

3. Global client state (auth user, theme, permission): ছোট হলে Context, জটিল/ঘনঘন বদলালে Redux Toolkit/Zustand
4. Context-এর সীমাবদ্ধতা: value বদলালে সব consumer re-render — high-frequency data-তে Context নয়
5. Redux Toolkit-ই এখন স্ট্যান্ডার্ড: `createSlice`, Immer-এ mutation-style update, boilerplate কম — খালি Redux লেখা অতীত
6. Form state আলাদা জগৎ — React Hook Form জাতীয় লাইব্রেরি; re-render কমিয়ে uncontrolled ভিত্তিতে চালায়

8. Performance Optimization: memo, useMemo, useCallback

Description: কখন এগুলো কাজ করে, কখন শুধু কোড জটিল করে — "সব জায়গায় useMemo" নয়, মাপা optimization-এর কথা বলা।

৬টা Key Point:

1. `React.memo(Component)`: props shallow-equal হলে re-render skip — শুধু ভারী বা লম্বা লিস্টের item component-এ দরকার
2. `useMemo` দামি হিসাব cache করে, `useCallback` function reference স্থির রাখে — মূল ব্যবহার memo-করা child-এ stable props পাঠানো
3. Inline object/array/function প্রতি render-এ নতুন reference — memo child-এ পাঠালে memo বৃথা; এখানেই `useMemo/useCallback` লাগে
4. আগে প্রোফাইল (React DevTools Profiler) — কোন component কেন re-render হচ্ছে দেখে টার্গেটেড fix; ঢালাও memo-তে লাভের চেয়ে জটিলতা বেশি
5. গঠনগত সমাধান আগে: state নিচে নামানো (colocate), children composition — memo ছাড়াই re-render কমে
6. লম্বা লিস্ট আসল অস্ত্র virtualization (react-window/virtuoso) — ১০,০০০ row-তে memo নয়, viewport-এ যা আছে তাই render

৫. Custom Hooks — Logic Reuse-এর সঠিক উপায়

Description: Component-এ জট পাকানো logic কীভাবে reusable hook-এ বের করবেন। সিনিয়র হিসেবে নিজের লেখা hook-এর উদাহরণ দিতে পারা উচিত।

৬টা Key Point:

1. Custom hook = stateful logic-এর reuse; UI নয় — UI reuse হলে component, logic reuse হলে hook
2. প্রতিটা hook call-এর নিজস্ব state — দুই component একই hook ব্যবহার করলেও state আলাদা (logic শেয়ার হয়, state না)
3. কমন প্যাটার্ন: `useDebounce`, `useLocalStorage`, `usePermission`, `useFetch`/axios wrapper — প্রজেক্টে বাস্তব উদাহরণ বলা (Sokrio-তে `useAxiosInstance`)
4. Rules of hooks-এর কারণ: React call order দিয়ে state মেলায় — তাই condition/loop-এর ভেতরে hook নয়
5. ভালো hook design: return value ছোট ও স্পষ্ট (`{data, loading, error}` বা tuple) — ভেতরের জটিলতা লুকানো
6. টেস্টযোগ্যতা: hook আলাদা হলে `renderHook` দিয়ে UI ছাড়াই টেস্ট — logic-in-component-এর চেয়ে বড় সুবিধা

৬. Controlled vs Uncontrolled Component ও Form Handling

Description: Form React-এর সবচেয়ে দৈনন্দিন কাজ। দুই approach-এর trade-off আর বড় form-এ performance সমস্যার সমাধান জানতে চাওয়া হয়।

৬টা Key Point:

1. Controlled: value state-এ, প্রতি keystroke-এ setState — full control (validation, format), কিন্তু প্রতি টাইপে re-render
2. Uncontrolled: DOM-ই value রাখে, দরকারে ref দিয়ে পড়া — কম re-render, কম control
3. বড় form-এ controlled-everything ধীর — React Hook Form uncontrolled + register দিয়ে re-render প্রায় শূন্যে নামায়
4. Validation স্তর: field-level onBlur + submit-level; schema লাইব্রেরি (Zod/Yup) দিয়ে type-safe rule এক জায়গায়
5. Server error-ও form-এ ফেরত ম্যাপ করা (Laravel-এর 422 error bag → field error) — UX-এর জন্য জরুরি
6. File input সবসময় uncontrolled — value programmatically set করা যায় না; ref-ই একমাত্র পথ

৭. React Router ও SPA Architecture

Description: Route-ভিত্তিক structure, protected route, আর code splitting — একটা admin panel-স্কেল SPA কীভাবে সাজাবেন।

৬টা Key Point:

1. Route definition এক জায়গায় centralized (route config array) — ছড়ানো `<Route>`-এর চেয়ে maintain সহজ
2. Protected route: wrapper component auth/permission চেক করে redirect — permission-gated menu-ও একই উৎস থেকে (এক permission enum)
3. Route-level code splitting: `lazy()` + `Suspense` — admin panel-এ প্রতিটা বড় পেজ আলাদা chunk, initial load ছোট
4. URL-ই state-এর উৎস: filter/pagination query param-এ রাখা — reload/share করলে একই view; সব state মেমোরিতে রাখা ভাল
5. Nested route + `Outlet` দিয়ে layout ভাগ (sidebar/topbar একবার) — layout duplication এড়ানো
6. Data loading: route change-এ fetch + loading state; router-এর loader বা React Query — যেন navigation-এ পুরনো ডেটা ঝলক না দেয়

৮. React 18+ Features: Concurrent, Suspense, Transitions

Description: আধুনিক React-এর সাথে আপডেট আছেন কিনা যাচাইয়ের প্রশ্ন — নাম মুখস্থ নয়, কোন সমস্যা সমাধান করে সেটা বলা।

৬টা Key Point:

1. Concurrent rendering: React render মাঝপথে থামিয়ে জরুরি কাজ (টাইপিং) আগে করতে পারে — UI জমে যাওয়া কমে
2. `useTransition/startTransition`: ভারী state update-কে "non-urgent" চিহ্নিত করা — সার্চ টাইপ smooth, ফলাফল একটু পরে
3. `useDeferredValue`: দ্রুত বদলানো value-র একটা পিছিয়ে থাকা কপি — debounce-এর declarative বিকল্প
4. Suspense: data/code লোড হওয়া পর্যন্ত fallback — loading state boilerplate কমায়; lazy loading-এ বহুদিন ধরেই ব্যবহৃত
5. Automatic batching সর্বত্র (promise/timeout-এর ভেতরেও) — আগের চেয়ে কম render, কিছু পুরনো অনুমান ভাঙতে পারে
6. দিগন্তে: Server Components (Next.js-এ মূলধারা) — server-only component-এর JS bundle-এ যায় না; ধারণাটা বলতে পারা যথেষ্ট

৯. API Integration Pattern ও Error Handling

Description: Axios/fetch দিয়ে API লেয়ার কীভাবে গোছাবেন — interceptor, token refresh, global error handling। দৈনন্দিন admin panel কাজের মেরুদণ্ড।

৬টা Key Point:

1. API call component-এ ছড়ানো নয় — service layer (`services/userService.ts`) বা query hook-এ কেন্দ্রীভূত
2. Axios instance + interceptor: request-এ token attach, response-এ global error handle — এক জায়গায়
3. 401 handling: interceptor-এ ধরে refresh/logout + redirect — প্রতিটা call-এ আলাদা চেক নয়
4. Error-এর স্তর ভাগ: validation error (form-এ দেখানো), expected error (toast), unexpected (generic message + Sentry log)
5. Race/cancel: পেজ ছেড়ে গেলে AbortController-এ pending request বাতিল — memory leak ও ভুল state ঠেকায়
6. Loading/empty/error তিনটা state-ই design করা — শুধু happy path বানানো জুনিয়র লক্ষণ; skeleton > spinner

১০. Component Design ও Testing Strategy

Description: Reusable component কীভাবে ডিজাইন করবেন আর কী টেস্ট করবেন — architecture-চিন্তা আর quality-চিন্তা একসাথে যাচাইয়ের প্রশ্ন।

৬টা Key Point:

1. Presentational vs container ভাগ এখনো দরকারি ধারণা: dumb component props নেয়, smart component data জোগায় — টেস্ট ও reuse সহজ
2. Composition (`children`, slot props) inheritance-এর জায়গায় — `<Modal>` ভেতরে কী থাকবে জানে না, জানার দরকারও নেই
3. Base component লেয়ার (BaseButton, BaseTable, BaseModal) — design consistency + লাইব্রেরি বদলালে এক জায়গায় বদল (Sokrio-তে এই প্যাটার্নই)
4. Prop drilling ও লেভেল ছাড়ালে সংকেত — composition বা context ভাবা; তবে সব prop-ই খারাপ না, explicit data flow-ও একটা গুণ
5. টেস্ট (Vitest + Testing Library): user-এর চোখে টেস্ট — "render হলে কী দেখা যায়, click-এ কী ঘটে"; implementation detail (state/internal call) টেস্ট নয়
6. টেস্ট অগ্রাধিকার: জটিল logic-ওয়ালা hook/util > critical flow (form submit, permission gate) > সব component snapshot — শেষটা প্রায় মূল্যহীন